MANUAL PRACTICO

Como subir un proyecto Python Flask (HTML y CSS) a GitHub desde Visual Studio Code

Commit, push, pull y creacion de la rama **develop**, paso a paso

Incluye: instalacion y configuracion, estructura del proyecto, .gitignore, autenticacion, flujo de ramas, resolucion de conflictos, referencia de comandos y solucion de errores comunes.

Contenido

1. Introduccion y requisitos **4**

1.1 Que es cada herramienta 4

1.2 Que necesitas antes de empezar 4

1.3 Conceptos clave (vocabulario basico) 4

2. Instalacion y configuracion inicial **5**

2.1 Instalar Git 5

2.2 Abrir la terminal dentro de VS Code 5

2.3 Configurar tu identidad en Git 5

2.4 Extensiones recomendadas de VS Code 6

3. Preparar el proyecto Flask **6**

3.1 Estructura tipica de un proyecto 6

3.2 Ejemplo minimo de app.py 6

3.3 Crear el entorno virtual y requirements.txt 7

3.4 Crear el archivo .gitignore (muy importante) 7

3.5 Crear un README.md 8

4. Crear el repositorio en GitHub **8**

5. Autenticacion con GitHub **9**

5.1 Opcion A: Iniciar sesion desde VS Code (la mas facil) 9

5.2 Opcion B: Personal Access Token (PAT) 9

5.3 Opcion C: Llave SSH (avanzado) 10

6. Subir el proyecto por primera vez **10**

6.1 Metodo grafico (panel Source Control) 10

6.2 Metodo con terminal (paso a paso) 10

7. Crear y usar la rama develop **12**

7.1 Crear la rama develop (terminal) 12

7.2 Publicar develop en GitHub 12

7.3 Crear la rama desde la interfaz de VS Code 13

7.4 Cambiar entre ramas 13

7.5 Establecer develop como rama por defecto (opcional) 13

8. Flujo de trabajo diario: modificar, commit, push y pull **13**

8.1 Pull: traer cambios antes de empezar 13

8.2 Ejemplo practico: agregar una pagina nueva . . . 14
8.3 Hacer lo mismo desde la interfaz de VS Code . . . 14
8.4 Como escribir buenos mensajes de commit . . . 15
8.5 Ver el historial . . . 15
9. Pasar los cambios de develop a main **15**
9.1 Opcion A: Merge local con terminal . . . 15
9.2 Opcion B: Pull Request en GitHub (recomendada) . . . 15
9.3 Mantener develop al dia con main . . . 16
10. Resolucion de conflictos **16**
10.1 Como se ve un conflicto . . . 16
10.2 Resolverlo en VS Code . . . 16
10.3 Como evitar conflictos . . . 17
11. Clonar el proyecto en otra computadora **17**
12. Referencia rapida de comandos **17**
12.1 Atajos utiles de VS Code . . . 18
13. Solucion de errores comunes **18**
13.1 Como deshacer cosas (con cuidado) . . . 19
14. Buenas practicas **20**
14.1 Flujo avanzado: ramas por funcionalidad (opcional) . . . 20
14.2 Resumen visual del flujo de ramas . . . 20
15. Lista de verificacion final **21**

1. Introduccion y requisitos

Este manual te guia desde cero para subir un proyecto web hecho con **Python, Flask, HTML y CSS** a un repositorio de **GitHub**, usando **Visual Studio Code** (VS Code). Aprenderas a hacer commits, push, pull y a trabajar con una rama de desarrollo llamada **develop**.

> **Nota:** En tu mensaje escribiste "Visual Basic Code". El editor correcto se llama **Visual Studio Code** (VS Code), es gratuito y es el que se usa en este manual. No confundir con Visual Basic, que es un lenguaje distinto.

1.1 Que es cada herramienta

Herramienta	Para que sirve
Git	Sistema de control de versiones. Guarda el historial de cambios de tu proyecto en tu computadora.
GitHub	Sitio web que aloja tus repositorios en la nube, para respaldo y colaboracion.
VS Code	Editor de codigo con panel de Git integrado y terminal.
Flask	Framework de Python para crear aplicaciones web.

1.2 Que necesitas antes de empezar

- Una computadora con Windows, macOS o Linux.
- Python 3 instalado (verifica con `python --version`).
- Visual Studio Code instalado (code.visualstudio.com).
- Git instalado (git-scm.com).
- Una cuenta gratuita en GitHub (github.com).
- Tu proyecto Flask ya funcionando en una carpeta local.

1.3 Conceptos clave (vocabulario basico)

Termino	Significado
Repositorio (repo)	Carpeta de proyecto controlada por Git. Puede ser local o remoto (GitHub).
Commit	Una "foto" guardada de tus cambios, con un mensaje que explica que hiciste.
Stage / Staging	Zona intermedia donde eliges que cambios entraran en el proximo commit (*git add*).
Push	Enviar tus commits locales a GitHub.
Pull	Traer y combinar a tu computadora los cambios que hay en GitHub.

Termino	Significado
Branch (rama)	Linea paralela de trabajo. Permite probar cosas sin danar la version estable.
main	Rama principal: contiene la version estable del proyecto.
develop	Rama de desarrollo: aqui se integran y prueban los cambios nuevos antes de pasarlos a main.
Remote / origin	Direccion del repositorio en GitHub. Por convencion se llama *origin*.
Merge	Unir los cambios de una rama dentro de otra.

2. Instalacion y configuracion inicial

2.1 Instalar Git

- **Windows:** descarga el instalador desde git-scm.com y deja las opciones por defecto (siguiente, siguiente, instalar).
- **macOS:** abre la Terminal y ejecuta `git --version`; si no esta instalado, macOS ofrecera instalarlo. Tambien puedes usar `brew install git`.
- **Linux (Debian/Ubuntu):** `sudo apt install git`.

Para comprobar la instalacion, abre una terminal y escribe:

Terminal

```
git --version
```

Debe mostrar algo como `git version 2.45.0`.

2.2 Abrir la terminal dentro de VS Code

En VS Code abre el menu **Terminal > New Terminal** (o presiona `Ctrl + Ñ` / `Ctrl + ``). La terminal se abre ya ubicada en la carpeta de tu proyecto.

2.3 Configurar tu identidad en Git

Git necesita saber quien eres para firmar tus commits. Usa el mismo correo que tienes en GitHub. Este paso se hace una sola vez por computadora.

Terminal

```
git config --global user.name "Tu Nombre"
git config --global user.email "tucorreo@ejemplo.com"
git config --global init.defaultBranch main
```

Verifica la configuracion:

Terminal

```
git config --list
```

Consejo: Si quieres que tu correo no sea publico, en GitHub ve a Settings > Emails y activa "Keep my email addresses private"; luego usa el correo noreply que GitHub te muestra.

2.4 Extensiones recomendadas de VS Code

- **Python** (Microsoft): ejecuta y depura codigo Python.
- **GitHub Pull Requests**: gestiona Pull Requests desde VS Code (opcional).
- **GitLens**: muestra quien cambio cada linea y el historial (opcional).

Git ya viene integrado en VS Code, por eso no necesitas una extension para los comandos basicos.

3. Preparar el proyecto Flask

3.1 Estructura tipica de un proyecto

Flask espera que los archivos HTML esten en una carpeta **templates** y los CSS, JS e imagenes en **static**. Un ejemplo:

Estructura de carpetas

```
mi_proyecto_flask/
|-- app.py                  <- archivo principal de Flask
|-- requirements.txt        <- lista de dependencias
|-- .gitignore              <- archivos que Git debe ignorar
|-- README.md               <- descripcion del proyecto
|-- .env                    <- variables secretas (NO se sube)
|-- venv/                   <- entorno virtual (NO se sube)
|-- templates/
|   |-- base.html
|   `-- index.html
`-- static/
    |-- css/
    |   `-- style.css
    |-- js/
    `-- img/
```

3.2 Ejemplo minimo de app.py

app.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

3.3 Crear el entorno virtual y requirements.txt

El entorno virtual (*venv*) aisla las librerias de tu proyecto. **Nunca se sube a GitHub**: en su lugar se sube **requirements.txt**, que permite a cualquiera reconstruirlo.

Terminal

```
# Crear el entorno virtual
python -m venv venv

# Activarlo en Windows (PowerShell)
venv\Scripts\Activate.ps1

# Activarlo en macOS / Linux
source venv/bin/activate

# Instalar Flask
pip install flask

# Guardar las dependencias en un archivo
pip freeze > requirements.txt
```

Atencion: Si en Windows PowerShell te da un error de "scripts deshabilitados", ejecuta una vez: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y vuelve a intentarlo. Tambien puedes usar CMD y el comando `venv\Scripts\activate.bat`.

3.4 Crear el archivo .gitignore (muy importante)

El archivo **.gitignore** le dice a Git que archivos NO debe subir. Es vital para no publicar contrasenas ni archivos pesados e innecesarios. Crea un archivo llamado exactamente `.gitignore` (con el punto al inicio) en la raiz del proyecto con este contenido:

.gitignore

```
# Entorno virtual
venv/
env/
.venv/

# Cache de Python
__pycache__/
*.pyc
*.pyo

# Variables de entorno y secretos
.env
*.env
instance/

# Base de datos local (si usas SQLite)
*.db
*.sqlite3

# Archivos de editores y sistema
.vscode/
.idea/
.DS_Store
Thumbs.db

# Logs
*.log
```

Atencion: Crea el .gitignore **ANTES** del primer commit. Si subes un archivo secreto por error, queda en el historial aunque lo borres despues. Si eso pasa, cambia de inmediato esas contrasenas o claves.

3.5 Crear un README.md

Es la portada de tu repositorio en GitHub. Un ejemplo sencillo:

README.md

```
# Mi Proyecto Flask

Aplicacion web hecha con Python, Flask, HTML y CSS.

## Instalacion
1. Clonar el repositorio
2. Crear entorno virtual: python -m venv venv
3. Activarlo e instalar: pip install -r requirements.txt
4. Ejecutar: python app.py
```

4. Crear el repositorio en GitHub

Primero crearemos el repositorio vacio en la nube. Sigue estos pasos:

- Entra a **github.com** e inicia sesion.

- Haz clic en el boton **+** (arriba a la derecha) y elige **New repository**.
- En **Repository name** escribe el nombre, por ejemplo `mi-proyecto-flask` (sin espacios ni acentos).
- Agrega una descripcion corta (opcional).
- Elige **Public** (cualquiera lo ve) o **Private** (solo tu y quien invites).
- **Importante:** NO marques "Add a README", "Add .gitignore" ni "Choose a license". Ya los tienes localmente y si los agregas aqui se generan conflictos en el primer push.
- Haz clic en **Create repository**.

GitHub mostrara una pagina con la URL de tu repositorio. Copiala; tendra una de estas formas:

URL del repositorio

```
HTTPS:  https://github.com/TU_USUARIO/mi-proyecto-flask.git
SSH:    git@github.com:TU_USUARIO/mi-proyecto-flask.git
```

Consejo: Si eres principiante, usa la URL **HTTPS**: es la mas sencilla de configurar.

5. Autenticacion con GitHub

Para subir codigo, GitHub debe saber que eres tu. GitHub ya **no acepta tu contrasena normal** en la linea de comandos. Tienes tres opciones:

5.1 Opcion A: Iniciar sesion desde VS Code (la mas facil)

La primera vez que hagas push o pull por HTTPS, VS Code o Git Credential Manager abriran una ventana del navegador para autorizar el acceso. Tambien puedes hacerlo manualmente:

- En VS Code, haz clic en el icono de **Cuentas** (esquina inferior izquierda, icono de persona).
- Elige **Sign in with GitHub to use GitHub Copilot / Sign in to Sync Settings** o cualquier opcion de GitHub.
- Se abrira el navegador; haz clic en **Authorize Visual-Studio-Code**.
- Regresa a VS Code: tu cuenta aparecera conectada.

5.2 Opcion B: Personal Access Token (PAT)

Si te pide usuario y contrasena en la terminal, en "contrasena" debes pegar un token:

- En GitHub: foto de perfil > **Settings** > **Developer settings** > **Personal access tokens** > **Tokens (classic)**.
- Clic en **Generate new token (classic)**.
- Ponle un nombre, una fecha de expiracion y marca el permiso **repo**.
- Clic en **Generate token** y **copia el token de inmediato** (solo se muestra una vez).
- Cuando Git pida la contrasena, pega el token.

Atencion: Trata el token como una contrasena: no lo compartas ni lo guardes dentro del proyecto.

5.3 Opcion C: Llave SSH (avanzado)

Terminal

```
ssh-keygen -t ed25519 -C "tucorreo@ejemplo.com"
# Presiona Enter para aceptar la ruta por defecto
# Luego copia el contenido de la clave publica:
cat ~/.ssh/id_ed25519.pub      # macOS/Linux
type %USERPROFILE%\.ssh\id_ed25519.pub   # Windows CMD
```

Pega esa clave en GitHub: **Settings > SSH and GPG keys > New SSH key**. Despues usa la URL SSH del repositorio.

6. Subir el proyecto por primera vez

Hay dos formas equivalentes: con la **interfaz grafica de VS Code** o con la **terminal**. Te explico ambas; aprender la terminal es muy recomendable porque es lo que usan los profesionales.

6.1 Metodo grafico (panel Source Control)

- Abre la carpeta de tu proyecto: **File > Open Folder** y selecciona la carpeta raiz (donde esta app.py).
- Haz clic en el icono de **Source Control** en la barra lateral izquierda (icono de ramas, atajo `Ctrl+Shift+G`).
- Pulsa el boton **Initialize Repository**. VS Code ejecuta *git init* y detecta todos los archivos como cambios (se marcan con una **U** de "Untracked").
- Revisa que **venv** y **__pycache__** NO aparezcan en la lista. Si aparecen, tu .gitignore esta mal ubicado o mal escrito.
- En la seccion **Changes**, pasa el cursor y haz clic en el **+** junto a "Changes" para hacer **stage** de todos los archivos (o el + individual de cada archivo).
- Escribe un mensaje en el cuadro **Message**, por ejemplo: `Primer commit: proyecto Flask inicial`.
- Pulsa el boton **Commit** (icono de check). Si pregunta si quieres hacer stage de todo automaticamente, elige **Yes**.
- Pulsa **Publish Branch**. VS Code te preguntara si quieres crear el repositorio en GitHub: puedes elegir esa opcion y VS Code lo crea por ti, o continuar con los pasos 6.2 si ya lo creaste.

Consejo: Cada icono tiene ayuda al pasar el cursor. Los colores de los archivos indican su estado: **U** = nuevo sin seguimiento, **M** = modificado, **A** = agregado al stage, **D** = eliminado.

6.2 Metodo con terminal (paso a paso)

Abre la terminal de VS Code en la raiz del proyecto y ejecuta los comandos en este orden:

Paso 1 - Inicializar el repositorio local

Terminal

```
git init
```

Crea una carpeta oculta `.git` donde Git guarda todo el historial.

Paso 2 - Ver el estado

Terminal

```
git status
```

Mostrara en rojo los archivos nuevos. Verifica que **venv/** no aparezca (si aparece, revisa el .gitignore).

Paso 3 - Agregar archivos al stage

Terminal

```
git add .            # agrega todo lo que no este ignorado
# o bien, archivos concretos:
git add app.py templates/ static/
```

Paso 4 - Hacer el primer commit

Terminal

```
git commit -m "Primer commit: proyecto Flask inicial"
```

Paso 5 - Asegurar que la rama se llame main

Terminal

```
git branch -M main
```

Paso 6 - Conectar con GitHub (agregar el remoto)

Terminal

```
git remote add origin https://github.com/TU_USUARIO/mi-proyecto-flask.git
```

Verifica que quedo bien:

Terminal

```
git remote -v
```

Paso 7 - Subir (push) por primera vez

Terminal

```
git push -u origin main
```

La opcion `-u` vincula tu rama local con la remota, asi despues solo escribes `git push` y `git pull`. Si Git te pide autenticacion, usa uno de los metodos de la seccion 5.

Actualiza la pagina de tu repositorio en GitHub: ya deberias ver todos tus archivos.

> **Nota:** Si ya creaste el repositorio y ves el mensaje *"remote origin already exists"*, usa `git remote set-url origin URL_NUEVA` para cambiar la direccion.

7. Crear y usar la rama develop

Trabajar directamente en **main** es arriesgado: si algo se rompe, tu version estable se daña. Por eso se usa un flujo con dos ramas permanentes:

Rama	Proposito	Regla
main	Version estable, lista para produccion o entrega.	Solo recibe cambios ya probados desde develop.
develop	Rama de integracion y trabajo diario.	Aqui se hacen y prueban los cambios nuevos.

7.1 Crear la rama develop (terminal)

Terminal

```
git checkout -b develop
# Forma moderna equivalente:
git switch -c develop
```

El comando crea la rama y te cambia a ella automaticamente. Comprueba en que rama estas:

Terminal

```
git branch
```

La rama activa aparece con un asterisco: `* develop`.

7.2 Publicar develop en GitHub

Terminal

```
git push -u origin develop
```

Ahora en GitHub, el selector de ramas (arriba a la izquierda de la lista de archivos) mostrara **main** y **develop**.

7.3 Crear la rama desde la interfaz de VS Code

- Haz clic en el nombre de la rama actual en la **barra de estado** (esquina inferior izquierda, dice *main*).
- En el menu superior elige **Create new branch...**.
- Escribe `develop` y presiona Enter. VS Code se cambia a la rama nueva.
- Pulsa **Publish Branch** en el panel Source Control para subirla a GitHub.

7.4 Cambiar entre ramas

Terminal

```
git checkout main        # ir a main
git checkout develop     # ir a develop
# o con la forma moderna:
git switch main
git switch develop
```

En VS Code basta con hacer clic en la rama de la barra de estado y elegir a cual ir.

> **Atencion:** Antes de cambiar de rama, haz commit de tus cambios o Git podria negarse a cambiar. Siempre revisa en que rama estas antes de editar.

7.5 Establecer develop como rama por defecto (opcional)

En GitHub: **Settings > Branches > Default branch** y selecciona develop. Asi los Pull Requests y clones apuntaran a develop.

8. Flujo de trabajo diario: modificar, commit, push y pull

Este es el ciclo que repetiras cada vez que trabajes en tu proyecto. Memoriza el orden:

Ciclo diario

```
1. git switch develop        -> ubicarte en develop
2. git pull                  -> traer lo ultimo de GitHub
3. (editar archivos en VS Code)
4. git status                -> ver que cambiaste
5. git add .                 -> preparar los cambios (stage)
6. git commit -m "mensaje"   -> guardar la version
7. git push                  -> subir a GitHub
```

8.1 Pull: traer cambios antes de empezar

Terminal

```
git switch develop
git pull
```

git pull descarga los cambios de GitHub y los combina con tu rama. Hazlo siempre antes de empezar a trabajar, sobre todo si usas varios equipos o trabajas con otras personas. En VS Code puedes usar el menu **(...) > Pull** del panel Source Control o el icono de sincronizacion de la barra de estado.

> **Consejo: git fetch** solo descarga la informacion sin modificar tus archivos; **git pull** equivale a fetch + merge.

8.2 Ejemplo practico: agregar una pagina nueva

Supongamos que agregas una pagina "Acerca de". Editas **app.py**, creas **templates/about.html** y modificas **static/css/style.css**.

a) Ver que cambio:

Terminal

```
git status
```

Veras los archivos modificados (M) y nuevos (U). Para ver las lineas exactas que cambiaron:

Terminal

```
git diff                # cambios aun no agregados al stage
git diff --staged       # cambios ya en stage
```

b) Preparar y confirmar:

Terminal

```
git add .
git commit -m "Agregar pagina Acerca de con estilos"
```

c) Subir a GitHub:

Terminal

```
git push
```

8.3 Hacer lo mismo desde la interfaz de VS Code

- Edita y guarda tus archivos (`Ctrl+S`). Aparecera un numero en el icono de Source Control con la cantidad de archivos cambiados.

- Abre el panel Source Control. Haz clic en un archivo para ver una comparacion lado a lado (rojo = eliminado, verde = agregado).
- Haz clic en **+** para pasar al stage cada archivo (o el + general para todos).
- Escribe el mensaje de commit y pulsa **Commit**.
- Pulsa **Sync Changes** (o **Push**). El boton *Sync* hace pull y luego push.

8.4 Como escribir buenos mensajes de commit

- Describe **que** hiciste, en pocas palabras y en tiempo presente o infinitivo: *"Agregar formulario de contacto"*.
- Evita mensajes vagos como "cambios", "arreglos" o "asdf".
- Haz commits pequenos y frecuentes, cada uno con un solo proposito.
- Ejemplos buenos: *"Corregir error de ruta en /login"*, *"Mejorar estilos responsive del navbar"*.

8.5 Ver el historial

Terminal

```
git log --oneline            # lista corta de commits
git log --oneline --graph --all   # con grafico de ramas
```

9. Pasar los cambios de develop a main

Cuando develop ya esta probado y estable, se fusiona (*merge*) en main. Hay dos formas.

9.1 Opcion A: Merge local con terminal

Terminal

```
git switch main
git pull                      # asegura main actualizado
git merge develop             # une develop dentro de main
git push                      # sube main a GitHub
git switch develop            # vuelve a trabajar en develop
```

9.2 Opcion B: Pull Request en GitHub (recomendada)

Un **Pull Request** (PR) es una solicitud para fusionar una rama en otra. Permite revisar los cambios antes de unirlos.

- Haz push de tu rama develop.
- En GitHub aparece un aviso amarillo **Compare & pull request**; haz clic. Si no aparece, ve a la pestana **Pull requests > New pull request**.
- Configura: **base: main** (destino) <- **compare: develop** (origen).
- Escribe un titulo y descripcion de lo que cambia.

- Clic en **Create pull request**.
- Revisa la pestana **Files changed**. Si todo esta bien, pulsa **Merge pull request** y luego **Confirm merge**.
- En tu computadora, actualiza main con `git switch main` y `git pull`.

> **Consejo:** No borres la rama develop despues de fusionar: es una rama permanente. GitHub ofrece borrar la rama del PR; no lo hagas en este caso.

9.3 Mantener develop al dia con main

Si main recibio cambios (por ejemplo, una correccion urgente), actualiza develop:

Terminal

```
git switch develop
git merge main
git push
```

10. Resolucion de conflictos

Un **conflicto** ocurre cuando dos versiones modificaron las mismas lineas de un archivo y Git no sabe cual conservar. No es un error grave: solo debes elegir.

10.1 Como se ve un conflicto

Git marca el archivo asi:

templates/index.html

```
<<<<<<< HEAD
<h1>Bienvenido a mi sitio</h1>
=======
<h1>Hola, bienvenido</h1>
>>>>>>> develop
```

- Lo que esta entre `<<<<<<< HEAD` y `=======` es **tu version actual**.
- Lo que esta entre `=======` y `>>>>>>>` es la **version entrante**.

10.2 Resolverlo en VS Code

- VS Code resalta el conflicto y muestra opciones encima: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes** y **Compare Changes**.
- Elige la opcion adecuada, o edita el texto manualmente y borra las marcas `<<<<<<<`, `=======` y `>>>>>>>`.
- Guarda el archivo.
- Haz stage del archivo resuelto y haz commit:

Terminal

```
git add templates/index.html
git commit -m "Resolver conflicto en index.html"
git push
```

Consejo: Si te asustas en medio de un merge, puedes cancelarlo con `git merge --abort` y volver al estado anterior.

10.3 Como evitar conflictos

- Haz **git pull** siempre antes de empezar a trabajar.
- Haz commits pequenos y frecuentes.
- Comunicate con tu equipo sobre quien edita que archivos.

11. Clonar el proyecto en otra computadora

Para descargar tu proyecto desde GitHub en otro equipo:

Terminal

```
git clone https://github.com/TU_USUARIO/mi-proyecto-flask.git
cd mi-proyecto-flask
git switch develop

python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # macOS / Linux
pip install -r requirements.txt
python app.py
```

En VS Code tambien puedes usar **Ctrl+Shift+P**, escribir **Git: Clone**, pegar la URL y elegir la carpeta destino.

12. Referencia rapida de comandos

Comando	Que hace
`git init`	Crea un repositorio nuevo en la carpeta actual.
`git clone URL`	Descarga un repositorio de GitHub.
`git status`	Muestra el estado de los archivos.
`git add .`	Agrega todos los cambios al stage.
`git add archivo`	Agrega un archivo concreto al stage.
`git commit -m "msg"`	Guarda los cambios del stage con un mensaje.

Comando	Que hace
`git push`	Sube commits a GitHub.
`git push -u origin rama`	Sube una rama nueva y la vincula.
`git pull`	Trae y fusiona cambios de GitHub.
`git fetch`	Descarga cambios sin fusionarlos.
`git branch`	Lista las ramas locales.
`git branch -a`	Lista ramas locales y remotas.
`git switch -c rama`	Crea una rama y cambia a ella.
`git switch rama`	Cambia de rama.
`git merge rama`	Fusiona la rama indicada en la actual.
`git log --oneline`	Muestra el historial resumido.
`git diff`	Muestra los cambios linea por linea.
`git remote -v`	Muestra las URLs remotas configuradas.
`git restore archivo`	Descarta cambios sin commit de un archivo.
`git restore --staged archivo`	Saca un archivo del stage.
`git stash`	Guarda temporalmente cambios sin commit.
`git stash pop`	Recupera los cambios guardados con stash.

12.1 Atajos utiles de VS Code

Atajo	Accion
`Ctrl + Shift + G`	Abrir el panel Source Control.
`Ctrl + Shift + P`	Paleta de comandos (escribe "Git:" para ver todas las acciones).
`Ctrl + ` `	Abrir o cerrar la terminal integrada.
`Ctrl + S`	Guardar el archivo.
`Ctrl + Enter`	Hacer commit (con el cursor en el cuadro de mensaje).

13. Solucion de errores comunes

Error / mensaje	Causa y solucion
fatal: not a git repository	No estas dentro de una carpeta con Git. Ejecuta `git init` o entra a la carpeta correcta con `cd`.
git is not recognized / *command not found*	Git no esta instalado o no esta en el PATH. Reinstala Git y reinicia VS Code.
Author identity unknown	Falta configurar nombre y correo. Ejecuta los comandos `git config --global user.name` y `user.email` (seccion 2.3).
remote origin already exists	Ya hay un remoto. Usa `git remote set-url origin URL`.
Authentication failed / *Support for password authentication was removed*	GitHub no acepta contrasena. Usa inicio de sesion por navegador o un Personal Access Token (seccion 5).
failed to push some refs / *rejected (non-fast-forward)*	GitHub tiene cambios que tu no tienes. Haz `git pull`, resuelve conflictos si hay, y luego `git push`.
refusing to merge unrelated histories	Creaste el repo en GitHub con README y localmente otro historial. Usa `git pull origin main --allow-unrelated-histories`.
src refspec main does not match any	Aun no hiciste ningun commit, o tu rama se llama master. Haz un commit y ejecuta `git branch -M main`.
Your local changes would be overwritten	Tienes cambios sin guardar. Haz commit, o usa `git stash` antes de cambiar de rama o hacer pull.
Subi la carpeta **venv** por error	Agrega venv/ al .gitignore y ejecuta `git rm -r --cached venv`, luego commit y push.
El .gitignore no ignora un archivo ya subido	Git sigue archivos ya registrados. Ejecuta `git rm --cached archivo`, commit y push.
Subi un archivo con contrasenas	Cambia de inmediato esas contrasenas o claves (borrar el archivo no limpia el historial) y agrega el archivo al .gitignore.

13.1 Como deshacer cosas (con cuidado)

Terminal

```
# Corregir el mensaje del ULTIMO commit (si aun no hiciste push)
git commit --amend -m "Nuevo mensaje"

# Deshacer el ultimo commit pero conservar los cambios en los archivos
git reset --soft HEAD~1

# Descartar cambios de un archivo (NO se pueden recuperar)
git restore archivo.py

# Crear un commit nuevo que revierte uno anterior (seguro, ya publicado)
git revert ID_DEL_COMMIT
```

Atencion: Evita `git reset --hard` y `git push --force` salvo que sepas exactamente lo que haces: pueden borrar trabajo de forma permanente.

14. Buenas practicas

- **Nunca subas secretos:** claves, contrasenas, tokens, SECRET_KEY de Flask o archivos .env. Usa variables de entorno.
- **No subas venv:** comparte solo requirements.txt. Actualizalo con `pip freeze > requirements.txt` cuando instales librerias nuevas.
- **No trabajes directo en main:** usa develop (o ramas por funcionalidad como `feature/login`).
- **Haz commits pequenos y descriptivos** con un solo proposito cada uno.
- **Haz pull antes de empezar** y push al terminar tu sesion de trabajo.
- **Escribe un README claro** para que otros (y tu en el futuro) sepan como ejecutar el proyecto.
- **Prueba antes de fusionar a main:** ejecuta la app y verifica que todo funcione.
- **Usa Pull Requests** para revisar cambios, incluso si trabajas solo.

14.1 Flujo avanzado: ramas por funcionalidad (opcional)

Cuando el proyecto crece, es comun crear una rama por cada funcionalidad, derivada de develop:

Terminal

```
git switch develop
git pull
git switch -c feature/formulario-contacto

# ... trabajar, add, commit ...

git push -u origin feature/formulario-contacto
# Abrir un Pull Request: feature/formulario-contacto  ->  develop
```

14.2 Resumen visual del flujo de ramas

Esquema

```
main      o----------------------o--------------------o   (estable)
           \                    /                    /
develop     o---o---o---o---o---o---o---o---o---o---o       (integracion)
                 \       /
feature           o--o--o                                   (funcionalidad)
```

15. Lista de verificacion final

Marca cada punto para confirmar que todo quedo correcto:

Verificacion	Comando o lugar
Git instalado y configurado	`git --version` y `git config --list`
Existe .gitignore y excluye venv y .env	Archivo .gitignore en la raiz
requirements.txt actualizado	`pip freeze > requirements.txt`
Repositorio creado en GitHub (vacio)	github.com > New repository
Remoto origin configurado	`git remote -v`
Primer commit y push realizados	Archivos visibles en GitHub
Rama develop creada y publicada	`git branch -a` y selector de ramas en GitHub
Flujo diario dominado	pull > editar > add > commit > push
Merge de develop a main probado	Pull Request o `git merge`

Felicidades: con estos pasos tu proyecto Flask queda respaldado en GitHub, con control de versiones y un flujo de trabajo ordenado con las ramas main y develop.